

شرح مشروع Real-Time Banking Fraud Detection بالكامل

مرجع شامل لكل جزء في المشروع: الفكرة، الأدوات، الكود، الأوامر، وإزاي كل حاجة اتربطت ببعض.

1. الفكرة العامة (في جملتين)

بنك عنده معاملات مالية بتحصل كل لحظة (إيداع، سحب، تحويل). المشروع بيبنى نظام بيراقب المعاملات دي لحظة بلحظة، يكتشف اللي شكلها مشبوه (فراود) أول ما تحصل، ويعرض النتائج في داشبورد مرئي (Power BI) بدل ما ننتظر تقرير يومي.

الاسم التقني للفكرة دي: **Real-Time Data Pipeline** (خط أنابيب بيانات لحظي) — يعني البيانات بتتحرك من مصدرها لحد ما تظهر كتقرير، من غير توقف، أول بأول.

2. الأدوات المستخدمة واللغات

الأداة	نوعها	اللغة اللي كتبنا بيها كود	دورها في المشروع
Docker + docker-compose	منصة تشغيل	YAML (ملف تعريف)	تشغيل كل الأدوات كـ "حاويات" منفصلة ومتصلة ببعض
Python	لغة برمجة	Python	اللغة الأساسية لكل كود كتبناه (API + Spark + Producer)
FastAPI	مكتبة Python	Python	بناء الـ API (نقطة الدخول والخروج للبيانات)
Kafka	أداة جاهزة	(مفیش كود، بس أوامر إعداد)	نقل المعاملات لحظيًا من مصدرها لباقي النظام
Spark (PySpark)	أداة معالجة بيانات ضخمة	Python	تحليل المعاملات وكشف الفراود وتلخيص النتائج
Hadoop HDFS	نظام تخزين موزع	(مفیش كود، بس أوامر إعداد)	تخزين كل المعاملات بشكل دائم
Power BI	برنامج تصور بيانات	مفیش كود خالص (واجهة رسومية)	عرض النتائج كرسومات بيانية

الخلاصة: اللغة الأساسية المستخدمة في كتابة الكود هي Python بالكامل. مفیش سطر واحد C أو Java اتكتب في المشروع (Kafka و Hadoop و Spark) نفسهم مكتوبين بلغات ثانية من مصممين الأصليين، لكن إحنا استخدمناهم كأدوات جاهزة وكتبنا الكود اللي بيتحكم فيهم بلغة Python).

3. رحلة المعاملة الواحدة (خطوة بخطوة)

- [1] يُولد معاملة وهمية Producer
↓ (HTTP POST)
- [2] Kafka تستقبلها وتبعثها لـ API
↓
- [3] bank-transactions تحمل المعاملة في "قناة" اسمها Kafka
↓
- [4] Spark Streaming ببسّم القناة، يفحص المعاملة، يعلّم عليها فراود ولا لّا
↓
- [5] Spark (Parquet بصيغة) يخزّن النتيجة في HDFS
↓
- [6] يُلخص كل البيانات المخزّنة generate_reports.py نشغل (يدويًا)
↓
- [7] بسيطة JSON النتائج بتتحفظ كملفات
↓
- [8] endpoints بتقرأ الملفات دي وتقدمها كـ API
↓
- [9] ويرسمها endpoints يسحب من الـ Power BI

4. كل ملف كتبناه، وإيه اللي فيه بالضبط

4.1 (Python) producer/simulate_transactions.py

الوظيفة: يُولد معاملات بنكية وهمية ويبعتها لـ API.

أهم كود فيه:

```
python

def generate_transaction(fraud_chance=0.1):
    is_fraud = random.random() < fraud_chance
    amount = random.uniform(5000, 20000) if is_fraud else random.uniform(50, 3000)
    return {
        "account_id": random.choice(ACCOUNTS),
        "amount": round(amount, 2),
        "transaction_type": random.choice(TX_TYPES),
        "location": random.choice(LOCATIONS),
    }
```

الشرح: بيختار عشوائيًا هل المعاملة دي هتكون "فراود" (باحتمال 10%). لو فراود، يخليها بمبلغ كبير (5000-20000)، لو عادية يخليها بمبلغ صغير (50-3000). وده اللي فسّر إن النتيجة النهائية طلعت قريبة من 90% عادي و 10% فراود.

بعدها بيبعتها لـ API:

```
python
```

```
res = requests.post(API_URL, json=tx, timeout=5)
```

4.2 `api/main.py` (Python - مكتبة FastAPI)

الوظيفة: بوابة الدخول (استقبال المعاملات) وبوابة الخروج (تقديم النتائج).

جزء الاستقبال (Ingestion):

```
python

@app.post("/transaction")
def create_transaction(tx: Transaction):
    event = {
        "transaction_id": str(uuid.uuid4()),
        "account_id": tx.account_id,
        "amount": tx.amount,
        ...
    }
    producer.send(TOPIC_NAME, value=event)
```

الشرح: لما توصلها معاملة، بتولد لها رقم فريد (`transaction_id`)، وبعدين بتبعها لـ Kafka عن طريق مكتبة اسمها `kafka-python`.

جزء التقديم (Serving):

```
python

@app.get("/fraud-by-location")
def fraud_by_location():
    return read_report("fraud_by_location.json")
```

الشرح: بتقرأ ملف JSON جاهز (اللي Spark ولده) وترجعه للي طلبه (Power BI أو أي حد فتح الرابط في المتصفح).

4.3 `spark-jobs/fraud_detection_streaming.py` (Python - PySpark)

الوظيفة: يسمع Kafka طول الوقت، يفحص كل معاملة جديدة، ويخزنها في HDFS.

أهم كود فيه:

```
python
```

```

raw_stream = (
    spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "kafka:29092")
    .option("subscribe", "bank-transactions")
    .load()
)

flagged = transactions.withColumn(
    "is_fraud", when(col("amount") > 5000, True).otherwise(False)
)

query = flagged.writeStream.format("parquet")
    .option("path", "hdfs://namenode:8020/data/transactions/")
    .start()

```

الشرح: القاعدة اللي استخدمناها لكشف الفرود بسيطة: أي معاملة قيمتها أكبر من 5000 جنيه بتتعلّم كـ "مشبوهة". دي قاعدة تعليمية بسيطة (Rule-Based)، مش Machine Learning حقيقي — نقطة تقدر تذكرها كـ "تطوير مستقبلي" في العرض.

4.4 `spark-jobs/generate_reports.py` (Python - PySpark)

الوظيفة: يقرأ كل البيانات من HDFS، يعمل تجميعات إحصائية، يحفظها كـ JSON.

أهم كود فيه:

```

python

df = spark.read.parquet("hdfs://namenode:8020/data/transactions/")

fraud_by_location = (
    df.filter(col("is_fraud") == True)
    .groupBy("location")
    .agg(count("*").alias("fraud_count"))
)

save_json("fraud_by_location.json", [row.asDict() for row in fraud_by_location.

```

الشرح: ببسأل البيانات 4 أسئلة إحصائية:

1. كام معاملة مشبوهة في كل منطقة؟ (`fraud_by_location.json`)
2. مين أكثر 10 حسابات فيهم فرود؟ (`top_fraud_accounts.json`)
3. نسبة العادي للمشبوه إيه؟ (`fraud_vs_normal.json`)
4. كل المعاملات بتفاصيلها (`all_transactions.json`)

ملحوظة مهمة: الخطأ الأصلية كانت الجزئية دي تتعمل بأداة اسمها **Hive** (أداة استعلامات SQL على البيانات الضخمة). حاولنا نشغلها بس واجهتنا مشاكل تقنية في التهيئة (**bug** معروف في الصورة اللي استخدمناها) واستهلك وقت كبير جدًا من غير حل. عشان

كده حوّلنا لنفس الفكرة لكن باستخدام Spark مباشرة، وهي نفس النتيجة المنطقية (تلخيص البيانات) لكن بطريقة أضمن وأسرع. Hive لسه موجودة في المشروع كجزء من الـ Architecture (طبقة Storage) حتى لو مش شغالة فعليًا وقت العرض.

5. إزاي كل الأدوات دي اتربطت ببعض

الرابط اللي وصل كل حاجة هو **Docker + docker-compose**. ملف `docker-compose.yml` هو "خريطة" بتوصف كل أداة كـ "حاوية" (Container)، وبتحطهم كلهم على شبكة داخلية واحدة اسمها `bigdata-net`، وكل حاوية بتقدر تكلم الثانية بمجرد اسمها (زي `transaction-api`، `spark-master`، `namenode`، `kafka`) من غير ما تحتاج IP Address. مثال من الملف:

```
yaml
services:
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    networks:
      - bigdata-net

  spark-master:
    image: apache/spark:3.5.0
    networks:
      - bigdata-net
```

6. كل الأوامر اللي استخدمناها (بالترتيب والغرض)

أوامر التأكد الأساسية

```
bash
docker --version      # مثبت Docker التأكد إن
docker ps             # عرض الحاويات الشغالة
docker ps -a          # عرض كل الحاويات (شغالة وواقفة)
```

تشغيل وإيقاف المشروع كله

```
bash
docker-compose up -d   # تشغيل كل الخدمات في الخلفية
docker-compose down    # إيقاف وحذف كل الحاويات
docker-compose build --no-cache < اسم الخدمة # إعادة بناء خدمة معينة من الصفر
```

متابعة الخدمات (Logs)

```
bash
```

```
docker logs -f kafka-init          # متابعة تجهيز Kafka
docker logs -f hdfs-init           # متابعة تجهيز HDFS
docker logs -f producer            # متابعة المحاكاة وهي تتبع معاملات
docker logs --tail 40 <اسم الحاوية> # آخر 40 سطر من اللوج (للتشخيص)
```

تشغيل الـ Spark Jobs

bash

تشغيل التحليل اللحظي (يفضل شغل طول الوقت)

```
docker exec -it spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --total-executor-cores 1 \
  --conf spark.jars.ivy=/tmp/.ivy2 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0 \
  /opt/spark-jobs/fraud_detection_streaming.py
```

توليد التقارير (يشتغل ويخلص)

```
docker exec -it spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --total-executor-cores 1 \
  --conf spark.jars.ivy=/tmp/.ivy2 \
  /opt/spark-jobs/generate_reports.py
```

تشغيل وإعادة تشغيل المحاكاة

bash

```
docker-compose up -d producer          # تشغيل المحاكاة أول مرة
docker-compose up -d --force-recreate producer # تشغيلها ثاني (تولد 200 معاملة جديدة)
```

فحص البيانات مباشرة في HDFS

bash

```
docker exec namenode hdfs dfs -ls /data/transactions/ # عرض الملفات المخزنة
```

اختبار الـ API يدويًا

bash

```
curl -X POST http://localhost:8000/transaction \
  -H "Content-Type: application/json" \
  -d '{"account_id": "ACC0001", "amount": 7500, "transaction_type": "withdrawal"}
```

```

http://localhost:8000/fraud-by-location
http://localhost:8000/top-fraud-accounts
http://localhost:8000/fraud-vs-normal
http://localhost:8000/all-transactions
http://localhost:8000/docs (API توثيق تلقائي لكل الـ)
http://localhost:9870 (HDFS واجهة)
http://localhost:8080 (Spark Master واجهة)

```

7. من فين جبنا البيانات؟ (سؤال متوقع من المدرب)

البيانات مش حقيقية — هي بيانات محاكاة (Simulated Data) ولأدائها سكريبت (`simulate_transactions.py`) بنفسه بشكل عشوائي (مبالغ عشوائية، حسابات عشوائية من قائمة 20 حساب وهمي، ومناطق محددة مسبقًا: القاهرة، الإسكندرية، الجيزة، الأقصر، أسوان). دي طريقة شائعة جدًا في مشاريع Big Data التعليمية لأن البيانات البنكية الحقيقية سرية ومش متاحة، فبنعمل محاكاة واقعية بدلًا منها.

8. أسئلة متوقعة من المدرب وإجاباتها الجاهزة

س: ليه استخدمت Kafka بدل ما الـ API تكتب في HDFS مباشرة؟ ج: Kafka بتحمي النظام من فقدان البيانات لو حصل ضغط مفاجئ (زي آلاف المعاملات في ثانية واحدة)، وبتفصل بين مين ببيعت البيانات ومين بيعالجها، فلو Spark وقع مؤقتًا، البيانات متحفظة في Kafka ومش بتضيع.

س: إزاي بتكتشف الفراود بالظبط؟ ج: حاليًا بقاعدة بسيطة (أي معاملة أكبر من 5000 جنيه = مشبوهة)، وده نقطة انطلاق. في تطوير مستقبلي ممكن نستبدلها بموديل Machine Learning حقيقي باستخدام Spark MLlib يتعلم من أنماط أعمق (وقت المعاملة، تكرارها، مكانها الجغرافي).

س: ليه معملتش الجزء الخاص بـ Hive؟ ج: عملناه فعليًا وهو موجود في المشروع (`/hive-scripts`)، بس واجهتنا مشكلة في استقرار الأداة نفسها وقت التشغيل، فقررنا نضمن نجاح العرض باستخدام Spark بدل منها لنفس الوظيفة (التلخيص والاستعلام)، وده قرار هندسي شائع في المشاريع الحقيقية (Trade-off بين الوقت المتاح والاستقرار).

س: البيانات دي حقيقية؟ ج: لأ، دي بيانات محاكاة (Simulated) ولأدائها بسكريبت بايثون بنسبة فراود 10% مقصودة، عشان نختبر عليها منطق كشف الفراود.

9. ملخص سريع للحفظ (30 ثانية قبل ما تتكلم)

"المشروع بيراقب معاملات بنكية لحظيًا. سكريبت بايثون بيوّلد معاملات وهمية ويبيعها لـ API (بايثون - FastAPI)، الـ API تحطها في Kafka، وSpark (بايثون) بيسمّعها لحظيًا ويكشف المشبوه منها بقاعدة بسيطة، ويخزن كل حاجة في HDFS. سكريبت ثاني في Spark بيلخص البيانات دي في تقارير JSON، والـ API بتقدمها لـ Power BI اللي بيعرضها كرسومات بيانية. كل الأدوات دي شغالة جوه Docker متصلة ببعض على شبكة واحدة."